

NATIONAL RESEARCH UNIVERSITY
HIGHER SCHOOL OF ECONOMICS

Faculty of Computer Science
Bachelor's Programme "Data Science and Business Analytics"

**Software Project Report on the Topic:
Multifunctional Disease Identification Module**

Fulfilled by:

Student of the Group БПАД221
Zagoskin Egor Igorevich



(signature)

27.05.2024

(date)

Assessed by the Project Supervisor:

Borevsky Andrey Olegovich
Research Assistant
School of Data Analysis and Artificial Intelligence, HSE University

(signature)

(date)

Moscow 2024

Contents

Annotation	4
Аннотация	4
1 Introduction	5
1.1 Basic terms and definitions	5
1.2 Background	5
1.3 Plan of work	6
2 Literature Review	6
2.1 MedMNIST	6
2.1 Retinal OCT images	7
3 Work with MedMNIST	8
3.1 Testing augmentations	8
3.2 Testing basic models	8
3.3 Optuna and Knowledge distillation	9
3.4 Interpretation	12
3.5 FDR control	15
4 Work with Retinal OCT images	16
4.1 Introduction	16
4.2 Freeze tests	17
4.3 Augmentation and Dropout test	17
4.4 Choice of FC layers	17
4.5 Knowledge distillation	18
4.6 Statistics	18
4.7 FDR control	20
4.8 Interface	22

5 Conclusion	23
References	24
Applications	24

Annotation

This work consists of creating new and/or refining existing computer vision algorithms in the field of bioinformatics. Additionally, it is intended to carry out risk assessment of false predictions and interpretation of the constructed neural networks. The result of the project will be a lightweight neural network specialized for disease detection in a certain area, which will be then integrated into an AI-powered module detecting a wide range of diseases.

Аннотация

Данный проект заключается в создании новых и/или доработке имеющихся алгоритмов компьютерного зрения в области биоинформатики. Дополнительно предполагается провести оценку риска ложных предсказаний и интерпретацию построенных нейронных сетей. Результатом проекта будет легковесная нейросеть, специализирующаяся на детекции заболеваний в определённой области, которая в последствии будет встроена в AI-powered модуль для определения широкого перечня заболеваний.

Keywords

ML, CV, CNN, Bioinformatics, Knowledge Distillation, LRP, FDR

1 Introduction

1.1 Basic terms and definitions

Machine learning (ML) – field of Artificial Intelligence studying algorithms, which are able to “learn” on a given set of data and then work with unseen before data of the similar type.

Convolutional neural network (CNN) – type of neural networks based on the idea of consequently applying filters and combining their results in order to extract key features of input data. CNNs are able to take into account spatial structure of data and are mostly used for image processing.

Knowledge distillation – process of training a smaller model utilizing a pretrained large model. The resulting model is supposed to perform better than if the larger model was not used.

Layer-wise relevance propagation (LRP) – technique allowing layer by layer, from top to bottom assess impact of neurons on the output. On the final step of the propagation each element of input (in our case each pixel of an input image) will be assigned with a number representing its relevance to the result.

Optuna - hyperparameter optimization framework. Allows to efficiently check performance of ML models from a given class and choose the best one.

False discovery rate (FDR) - a proportion of type I errors. That is, $FDR = FP / (FP + TP)$, where FP is the number of false positives (negative samples classified as positive) and TP is the number of true positives (correctly classified positive samples).

1.2 Background

In the last decade Artificial Intelligence proved itself to be applicable in a wide range of tasks. For instance, its subfields, computer vision, is now used in industrial robots, surveillance systems, autonomous vehicles and others. One of the most promising applications of CV is processing of medical images of various nature (e.g. simple photographs, X-ray scans, ultrasonic images). Since this field is extremely young, but very broad, there exist a great number of problems which were not properly addressed yet and many tasks only have open-source solutions of quite poor quality.

This work is focused on improving the quality of existing solutions and development of new methods for disease detection problem. To diagnose a disease is a hard task which can now be effectively performed only by highly educated medical experts. Considering that the vast majority of the world population live in developing countries, there are billions of those who cannot get proper diagnoses. And even for those who can it sometimes may cost a fortune. AI-powered modules for disease detection could mitigate this problem by making diagnosing more accessible

and affordable. And if (or rather when) there are ML tools, which perform better than an average medical specialist, healthcare will be also of a better quality.

1.3 Plan of work

First, we tested different ML methods on one of MedMNIST [\[1\]](#) datasets. We began with training several basic CNN models and researching augmentations effect on results. Then, in order to exceed results of the basic models, we exploited Optuna [\[2\]](#). Using Optuna we found two models of different size, trained larger one and then trained smaller one using logits of the larger as an additional target as it described in the Pytorch Tutorial on knowledge distillation [\[4\]](#). This allowed us to get a high-quality lightweight model, which is less computationally expensive and more interpretable than large models.

After the small model is trained, it was interpreted using algorithms described in chapter “Layer-Wise Relevance Propagation: An Overview” of the book by Wojciech Samek et al. [\[5\]](#) Interpretation is especially crucial in biomedical field because the cost of mistake is extreme and it is much more preferable to understand why the machine made a certain decision instead of just using the result out of “black box”.

Then, since different types of errors in medical field have drastically different importance (for instance, False Positive might mean sending a patient on an unnecessary operation, while False Negative might only mean to say to a patient to come again in several months), we conducted statistical analysis of our model regarding the False Discovery Rate. This was done on the basis of the A. Borevsky’s Bachelor’s Thesis [\[6\]](#).

In the second part of the work, we applied the above methods to the Retinal OCT images dataset [\[7\]](#).

2 Literature Review

2.1 MedMNIST

MedMNIST is a collection of biomedical datasets ([Figure 1](#)) designed to support research and educational purposes. It consists of small size (28 x 28 for 2D images) images labeled for classification tasks. It consists of images pre-processed into a small size (28 x 28 for 2D images) in order to facilitate fast training. All images have attached classification labels so no background knowledge is required for users.

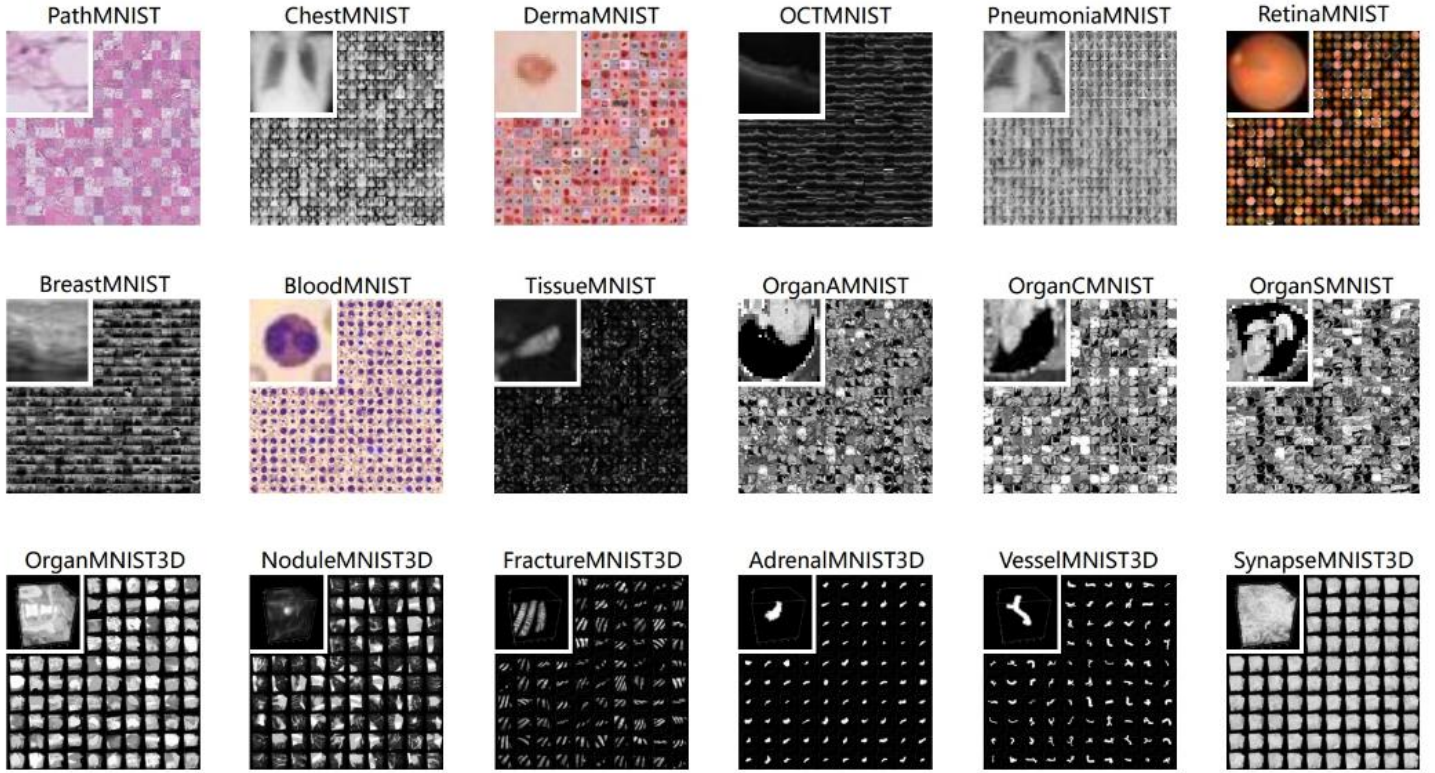


Figure 1 – Examples of datasets constituting MedMNIST.

Authors of the MedMNIST also provided standard train-validation-test splits and benchmarked several baseline methods (ResNets and AtuoML tools). One of the goals of our work is to outperform these baseline methods.

2.1 Retinal OCT images

Retinal OCT images is a dataset of optical coherence tomography images ([Figure 2](#)) of four categories, which are NORMAL, CNV, DME and DRUSEN (the last three are diseases: choroidal neovascularization, diabetic macular edema and drusen). The dataset is split into 61284 images for training, 7659 for validation and 7664 for testing. The size of the images varies from 496 x 512 x 3 to 512 x 1536 x 3.

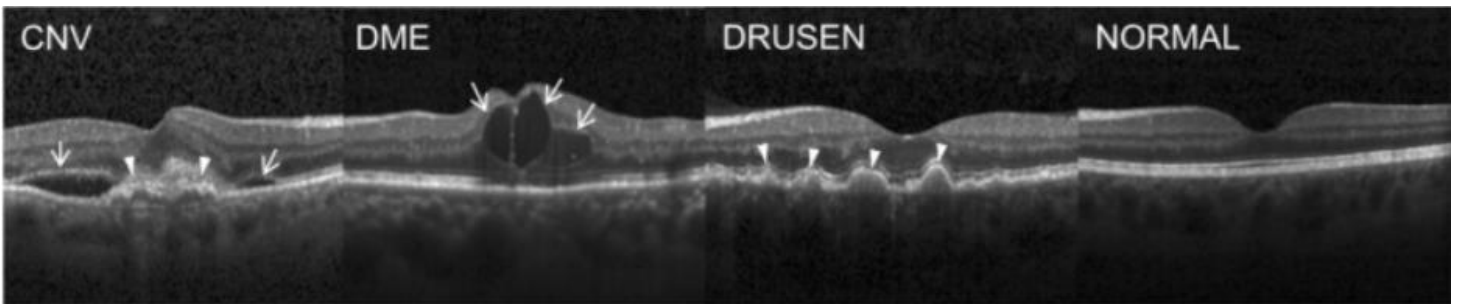


Figure 2 – Examples of Optical Coherence Tomography Images.

At the moment, there is only one work [8] on this dataset which is conducted by Kaggle user Mohamed Gobara. Mohamed trained Xception-based model based on 41% of the training data and achieved 96.1% accuracy. He also calculated precision, recall and f1-score for all classes and constructed a confusion matrix. We will consider this work as a benchmark and try to achieve greater accuracy using less data and lighter models.

3 Work with MedMNIST

3.1 Testing augmentations

For testing ML methods we choose PathMNIST dataset consisting of 89996 train, 10004 validation and 7180 test histological images of tissues and proposing a classification task with 9 labels. The authors of MedMNIST achieved on this dataset the best quality of AUC (area under the ROC Curve) = 0.990 and ACC (accuracy) = 0.911 using ResNet-50 (we take it as a benchmark).

First, we compared results of pre-trained ResNet-18 on the images without augmentations, images resized to 32 x 32 (natural for ResNet) and images with brightness, contrast and hue randomly shifted with ColorJitter. Results are presented in [Table 1](#) (for comparison we added the result which the authors got with ResNet-18). For all further experiments we exploited ColorJitter as it showed the best quality.

Table 1 – Comparison of augmentations.

	AUC	ACC
No augmentation	0.982	0.856
Resize	0.950	0.715
ColorJitter	0.989	0.886
Benchmark ResNet-18	0.983	0.907
Benchmark ResNet-50	0.990	0.911

3.2 Testing basic models

Second, we compared our ResNet-18 with three other models: DefaultNet, used in tutorial showing how to use MedMNIST (5 convolutional, 3 dense layers), FirstNet, created ourselves (7 convolutional, 3 dense layers), DropoutNet, which is FirstNet with added dropout layers. Results are presented in [Table 2](#). As we can see FirstNet outperformed Benchmark ResNet-50 (It is worth noting though, that FirstNet without ColorJitter do not exceed the benchmark), while dropout layers have a horrific effect on model's performance.

Table 2 – Comparison of several CNNs.

	AUC	ACC
ResNet-18	0.989	0.886
DefaultNet	0.989	0.887
FirstNet	0.993	0.916
DropoutNet	0.807	0.301
Benchmark ResNet-18	0.983	0.907
Benchmark ResNet-50	0.990	0.911

3.3 Optuna and Knowledge distillation

Third, we utilized Optuna in order to find models with even better performance. Optuna allows users to choose different samplers and pruners. Its documentation suggests (on the basis of [3]) to use RandomSampler with MedianPruner or TPESampler with HyperbandPruner for not very deep networks. In this work we chose TPESampler with HyperbandPruner since TPESampler is the default sampler for Optuna.

The aim was to find a large model with best possible performance and a small model which can be later improved with knowledge distillation from the large one. To use Optuna, we, first, defined the set of model architectures to be tested ([Figure 3](#)). Then, we created a function which generates a model, trains it, validates and reports the result (we used a combination of AUC and ACC as a metric) ([Figure 4](#)). Based on this function Optuna conducts a study, searching for the most effective architecture out of the defined.

```

1 def define_model(trial):
2     # We optimize the number of layers, hidden units and dropout ratio in each layer.
3     n_layers = trial.suggest_int("n_layers", 8, 9)
4     layers = []
5
6     in_features = 3
7     for i in range(1, n_layers + 1):
8         out_features = trial.suggest_int("n_units_l{}".format(i), 64, 256)
9         layers.append(nn.Conv2d(in_features, out_features, kernel_size=3, padding=1))
10        layers.append(nn.BatchNorm2d(out_features))
11        layers.append(nn.ReLU())
12        if i == 3 or i == 7:
13            layers.append(nn.MaxPool2d(kernel_size=2, stride=2))
14
15        in_features = out_features
16    layers.append(nn.AdaptiveAvgPool2d(output_size=(2, 2)))
17    layers.append(nn.Flatten())
18    out_features = trial.suggest_int("n_units_fc", 256, 512)
19    layers.append(nn.Linear(in_features * 4, out_features))
20    layers.append(nn.BatchNorm1d(out_features))
21    layers.append(nn.ReLU())
22    layers.append(nn.Linear(out_features, n_classes))
23
24    return nn.Sequential(*layers)

```

Figure 3 – An example of the function defining the set of architectures to try.

```

1 def objective(trial):
2     # Generation of a model and an optimizer
3     model = define_model(trial).to(device)
4     lr = trial.suggest_float("lr", 1e-5, 1e-2, log=True)
5     optimizer = optim.AdamW(model.parameters(), lr=lr)
6     scheduler = lr_scheduler.MultiStepLR(optimizer, milestones=[4, 7], gamma=0.1)
7
8     # Training of the model
9     for epoch in range(10):
10        model.train()
11        for batch_idx, (data, target) in enumerate(train_loader):
12            target = target.squeeze().long().to(device)
13            optimizer.zero_grad()
14            output = model(data.to(device))
15            loss = F.cross_entropy(output, target)
16            loss.backward()
17            optimizer.step()
18        scheduler.step()
19
20        # Validation of the model
21        model.eval()
22        y_val = torch.tensor([])
23        with torch.no_grad():
24            for batch_idx, (data, target) in enumerate(val_loader):
25                output = model(data.to(device))
26                output = output.softmax(dim=-1).cpu()
27                y_val = torch.cat((y_val, output), 0)
28        y_val = y_val.detach().numpy()
29        evaluator = Evaluator(data_flag, 'val')
30        metrics = evaluator.evaluate(y_val)
31        res = (metrics[0] + 2 * metrics[1]) / 3
32        trial.report(res, epoch)
33
34        # Handle pruning based on the intermediate value
35        if trial.should_prune():
36            raise optuna.exceptions.TrialPruned()
37    return metrics[1]

```

Figure 4 – An example of the objective function for the study.

As a result, we constructed LargeNet (8 convolutional, 2 dense layers) with 2,234,393 parameters and SmallNet (3 convolutional, 2 dense layers) with 129,903 parameters. Their performance (before and after the distillation for SmallNet) is presented in [Table 3](#). Both LargeNet and SmallNet clearly surpassed FirstNet, and although they differ in number of parameters more than 17 times, the difference between their results is less than 1% in accuracy, half of which is eliminated by knowledge distillation.

Table 3 – Comparison of Optuna-searched models, FirstNet and Benchmark ResNet-50.

	AUC	ACC
LargeNet	0.993	0.932
SmallNet	0.992	0.924
SmallNet after distillation	0.992	0.929
Benchmark ResNet-50	0.990	0.911
FirstNet	0.993	0.916

3.4 Interpretation

Fourth, we constructed LRP algorithm and used it to understand which pixels affected predicted class of the image the most.

In this algorithm we, first, construct a sequence of representations (maps) of a given image showing how the data of the image changes after going through each layer (that is the first map is the image itself and the last is the set of logits) ([Figure 5](#)). After that we assign to the logit corresponding to the predicted class relevance equal to its value (other logits have zero relevance) and propagate this relevance to the elements of the previous map proportional to their effect on the logit value. Then we propagate the relevance to the further maps until it reaches the first map which is the image itself ([Figure 6](#)).

So, as the output of LRP, we get a heatmap of the image’s pixels showing the effect of each pixel on the predicted class. Examples of images and heatmaps of their pixels’ relevance to the classification is shown on [Figure 7](#).

```

1 cur = 7
2 X = test_dataset[cur][0].reshape(1,3,28,28)
3 maps = [X]+[None]*n
4 for l in range(n):
5     maps[l+1] = layers[l].forward(maps[l])
6
7 scores = np.array(maps[-1].cpu().data.view(-1))
8 print('SOCRES')
9 for i in np.argsort(-scores):
10     print(str(i)+':', scores[i])
11 print('TRUE LABEL')
12 test_dataset[cur][1].item(), info['label'][str(test_dataset[cur][1].item())]

```

```

SOCRES
0: 2.8589318
5: 2.157882
4: -2.5797813
6: -2.6136775
7: -2.7612936
1: -2.9611695
8: -3.0953436
2: -4.2509685
3: -4.352945
TRUE LABEL
(0, 'adipose')

```

Figure 5 – code creating the maps and showing the scores (logits) of the model.

```

1 mask = torch.FloatTensor(((np.arange(9)==np.argmax(scores)).reshape([1,9,1,1])))
2 rels = [None]*n + [(maps[-1]*mask).data]
3 for l in range(n)[::-1]:
4     maps[l] = maps[l].detach().requires_grad_(True)
5     if l <= 6:
6         incr = lambda z: z + 0.5*(z**2).mean()**.5).detach()
7     if l >= 7:
8         incr = lambda z: z
9     if isinstance(layers[l],torch.nn.Conv2d) or isinstance(layers[l],torch.nn.AvgPool2d):
10        z = incr(layers[l].forward(maps[l])) + 1e-9
11        s = (rels[l+1]/z).detach()
12        (z*s).sum().backward()
13        c = maps[l].grad
14        rels[l] = maps[l].detach() * c
15    else:
16        rels[l] = rels[l+1]

```

Figure 6 – code conducting back propagation of the relevance.

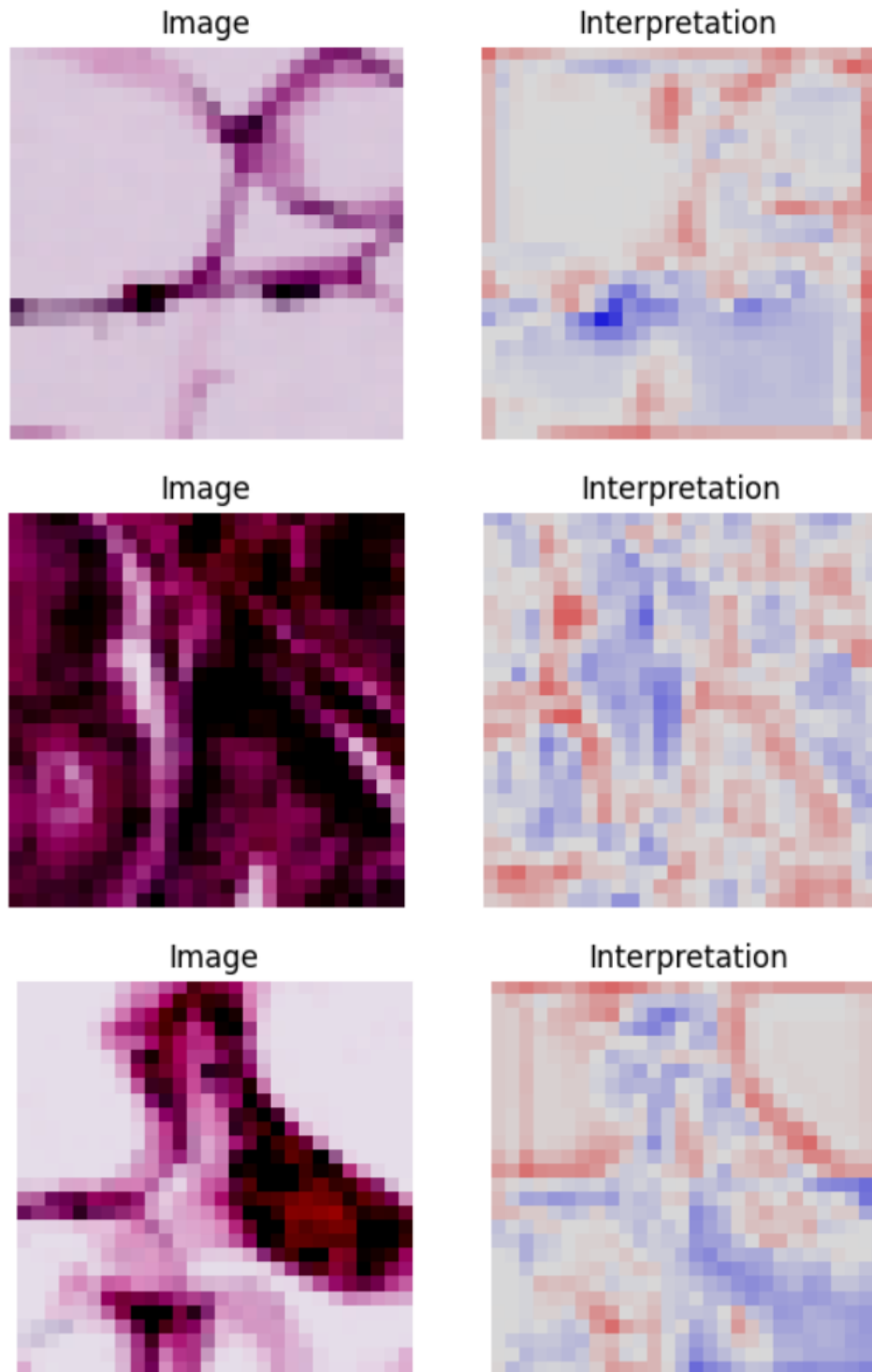


Figure 7 – Images of tissues (on the left) and relevance heatmaps (red pixels have positive impact on the logit corresponding to the predicted class, blue – negative) corresponding to this image (on the right).

3.5 FDR control

Finally, we arbitrary chose class ‘Adipose’ as positive and used Benjamini-Hochberg protocol (BHP) with empirical p-values (EPV) to construct the dependence between number of accepted discoveries and False Discovery Rate assuming that true labels are unknown ([Figure 8.1](#), [Figure 8.2](#)). Current model has FDR = 2.2%.

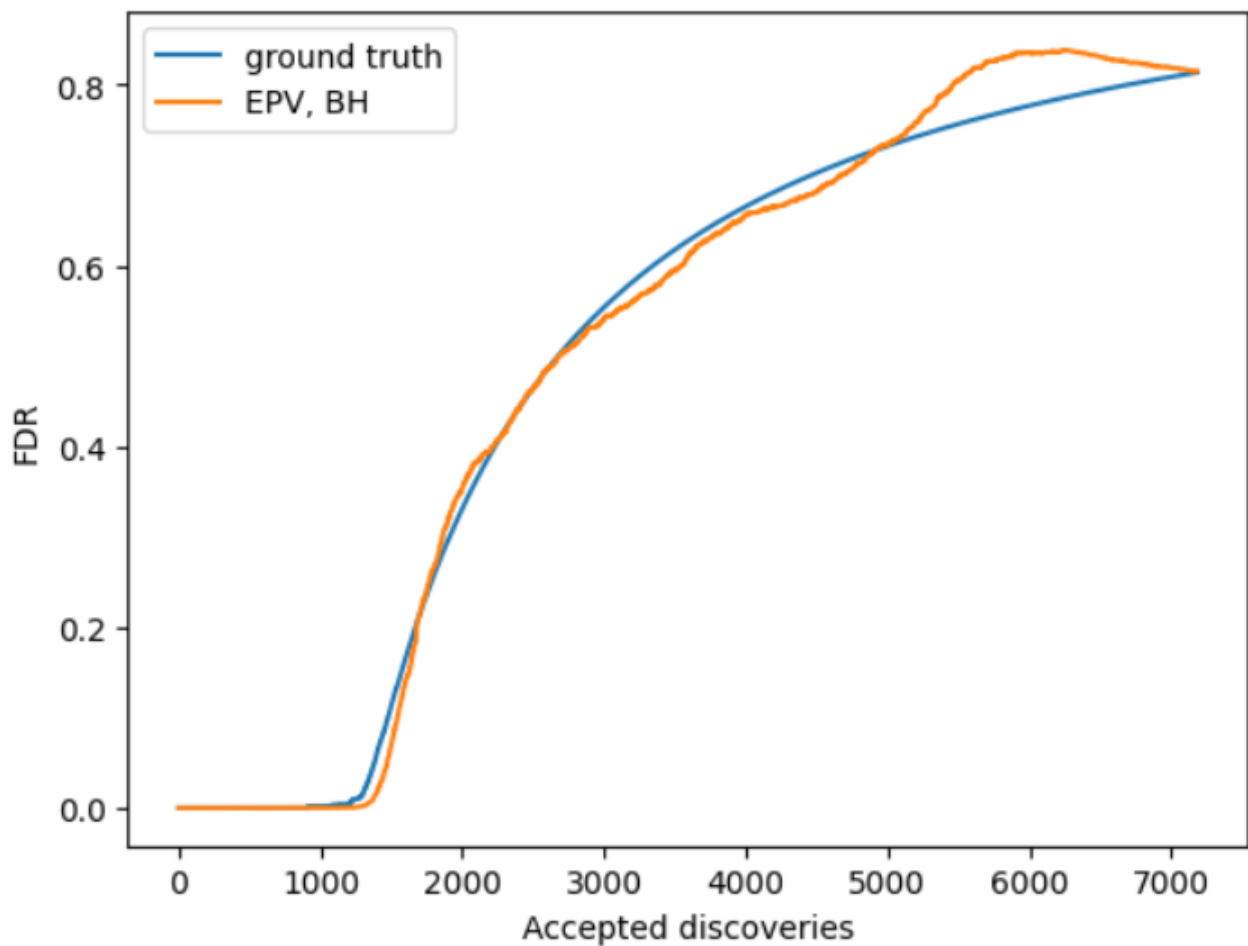


Figure 8.1 – Graph showing dependence between the number of accepted discoveries and FDR. Blue line represents FDR constructed with true labels. Orange line – with BHP and EPV.

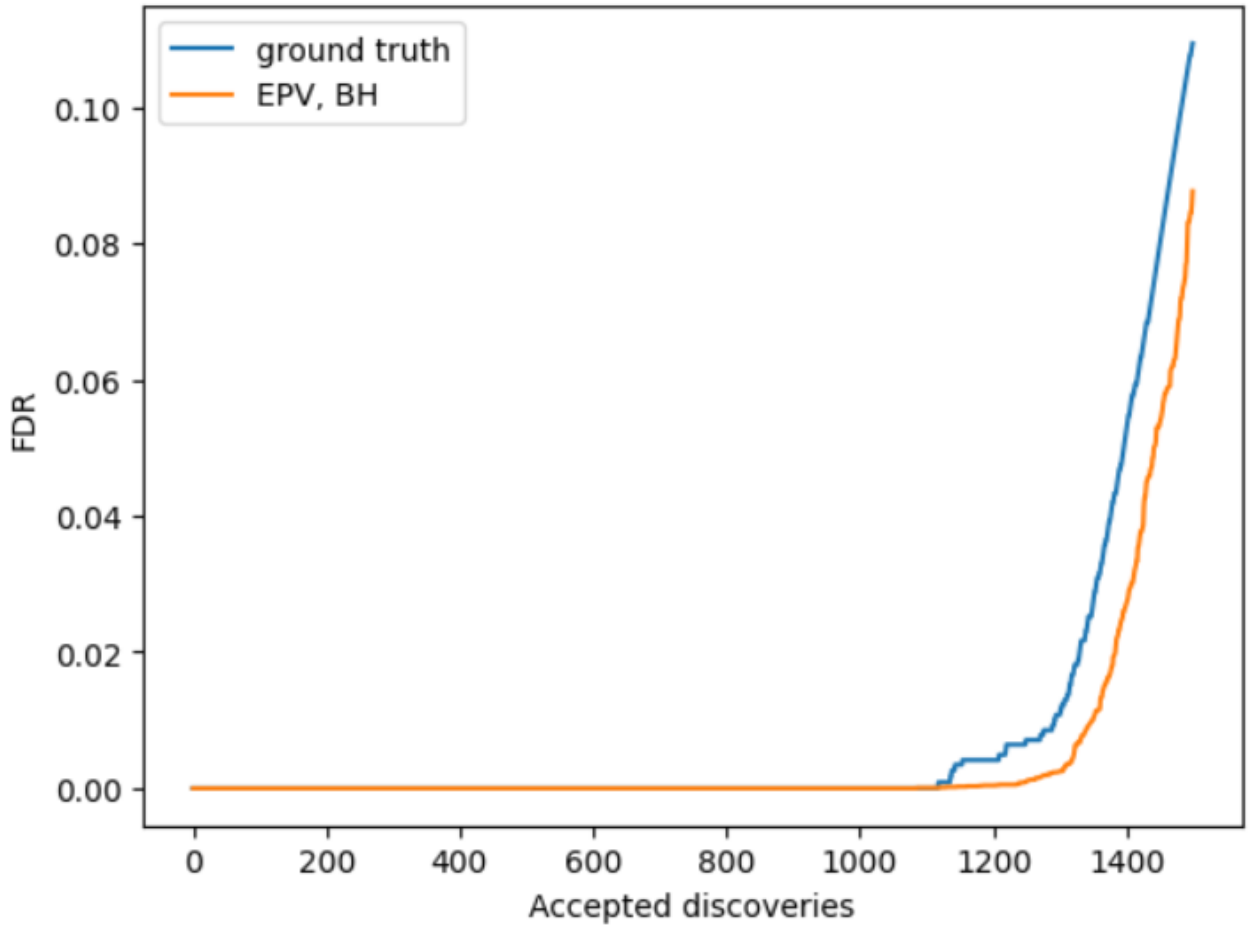


Figure 8.2 – Relevant part of the Figure 4.1 on a larger scale.

4 Work with Retinal OCT images

4.1 Introduction

The biggest challenge in work with this dataset is its large size. To address the problem of processing of 76 thousand high-resolution images on amateur equipment we had to be much more efficient than the rival work.

First, we resized all images to 496 x 512 x 3 and organized them into numpy arrays. Second, we noticed that for any image values of its 3 channels are identical and reduced the size to just 496 x 512. Third, we used an alternative way of supplying model with data to that proposed in the rival work (`ImageDataGenerator().flow()` instead of `ImageDataGenerator().flow_from_dataframe()`).

In addition, we had to limit our choice of models to those which prioritize the speed of learning, thus we were choosing from the various EfficientNets and alike. In the end we used EfficientNetV2-B3 pretrained on ImageNet dataset [9] which has 13 million parameters as opposed to 21 million of Xception and architecture designed for fast learning.

4.2 Freeze tests

First, we tested the number of frozen bottom layers which gives the best performance. Model architecture: EfficientNetV2-B3 and two fully connected layers with dropout (128 and 4 neurons) on top. We trained the model on tiny portion of 4000 images for 6 epochs. Batch size = 8.

As it can be seen in [Table 4](#), freezing of 110 layers showed the best result, so this is the level which we used in all next models.

Table 4 – Comparison of freezing levels.

	Accuracy
Freeze 0	0.903
Freeze 110	0.909
Freeze 212	0.891
Freeze 300	0.904

4.3 Augmentation and Dropout test

Then we tested how augmentation of brightness and inclusion of dropout affect the result. This time we used 24000 images (it is 39% of available train data), but trained models on each image only once. Batch size = 8.

It was concluded from [Table 5](#) that augmentation does not improve the result and dropout significantly worsen it, so both were not used anymore.

Table 5 – Comparison of models with and without Augmentation and Dropout.

	Accuracy
Augmentation and Dropout	0.940
Only Dropout	0.941
No Augmentation, No Dropout	0.959

4.4 Choice of FC layers

Next, we compared FC layers of different size. We tested two architectures with 2 FC layers (128 and 4 neurons, 256 and 4 neurons) and one architecture with 3 FC layers (192 and 32 and 4 neurons). 24000 images. Batch size = 8.

As it is shown in [Table 6](#), architecture with 128 neurons gives the best result.

Table 6 – Comparison of FC layers.

	Accuracy
128 and 4 neurons	0.959
256 and 4 neurons	0.958
192 and 32 and 4 neurons	0.939

4.5 Knowledge distillation

Large model: EfficientNetV2-B3 with two FC layers on top (128 and 4 neurons). 13 million parameters.

Small model: EfficientNetV2-B0 with two FC layers on top (128 and 4 neurons). 6 million parameters.

Now we train two models, Large and Small, on 24000 images and conduct knowledge distillation from the former to the latter. Number of epochs was determined by validation accuracy. For Large and Small it was 5 epochs, for Small with distillation 6 epochs. Batch size = 16. Results are presented in [Table 7](#).

Table 7 – Comparison of final models.

	Accuracy
Large Model	0.969
Small Model	0.965
Small Model with distillation	0.968

4.6 Statistics

Below are some statistics and confusion matrix of the Small Model with distillation ([Figure 9.1](#), [Figure 10](#)) and same statistics of the rival model ([Figure 9.2](#)). It can be seen that our model clearly outperforms the rival.

	precision	recall	f1-score	support
CNV	0.99	0.96	0.98	3165
DME	0.96	0.97	0.97	1099
DRUSEN	0.85	0.95	0.90	783
NORMAL	0.98	0.98	0.98	2617
accuracy			0.97	7664
macro avg	0.95	0.97	0.96	7664
weighted avg	0.97	0.97	0.97	7664

Accuracy of the Model: 96.8%

Figure 9.1 – Statistics of the Small Model with distillation.

	precision	recall	f1-score	support
CNV	0.99	0.96	0.97	3165
DME	0.94	0.98	0.96	1099
DRUSEN	0.83	0.94	0.88	783
NORMAL	0.98	0.97	0.98	2617
accuracy			0.96	7664
macro avg	0.94	0.96	0.95	7664
weighted avg	0.96	0.96	0.96	7664

Accuracy of the Model: 96.1%

Figure 9.2 – Statistics of the rival model.

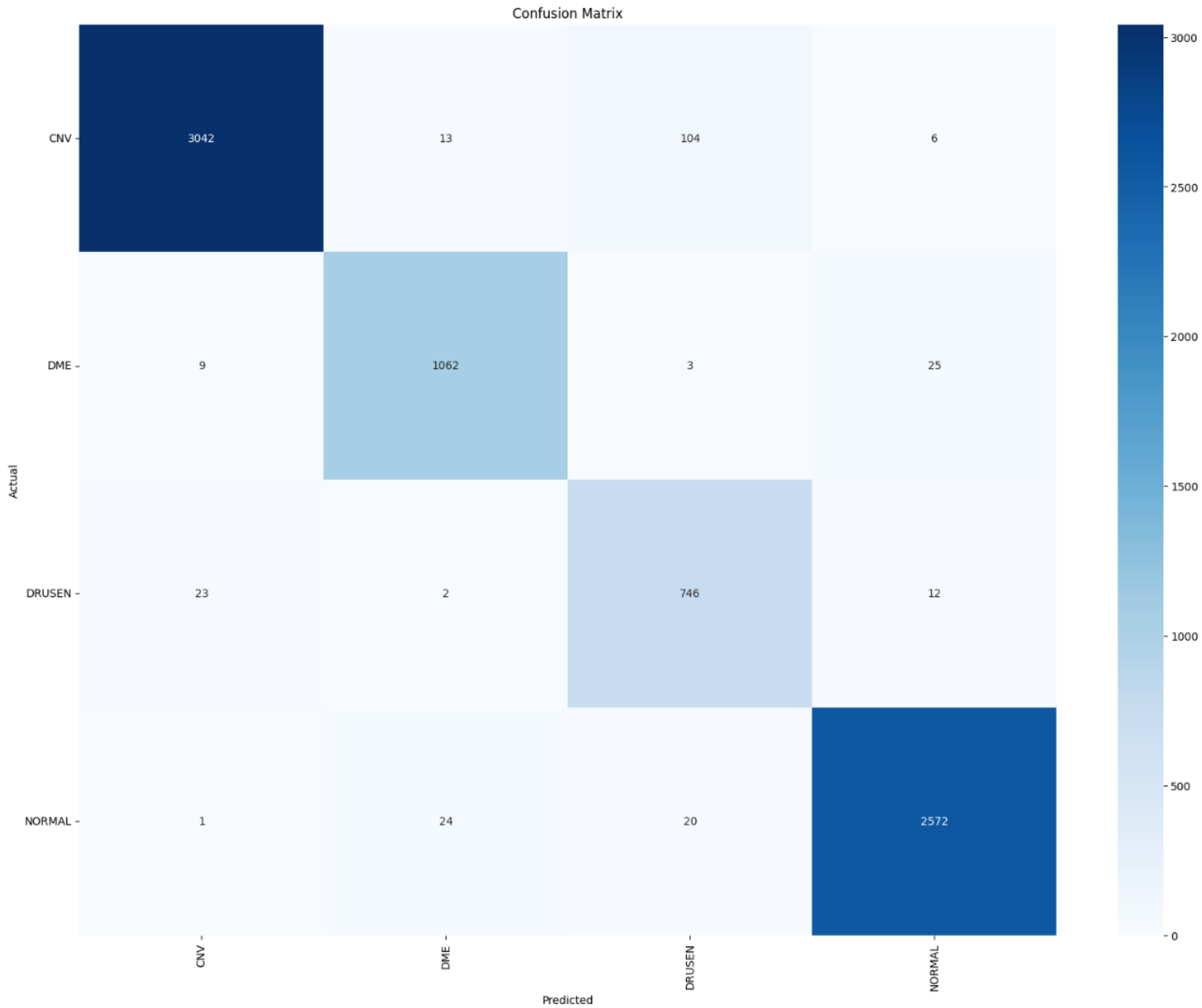


Figure 10 – Confusion matrix of the Small Model with distillation.

4.7 FDR control

Class NORMAL is chosen as negative and others as positives. Then we use BH protocol with empirical p-values to construct the dependence between number of accepted discoveries and False Discovery Rate assuming that true labels are unknown ([Figure 11.1](#), [Figure 11.2](#)). Current model has $\text{FDR} = 0.9\%$ which can be easily adjusted to desirable level using the constructed dependence.

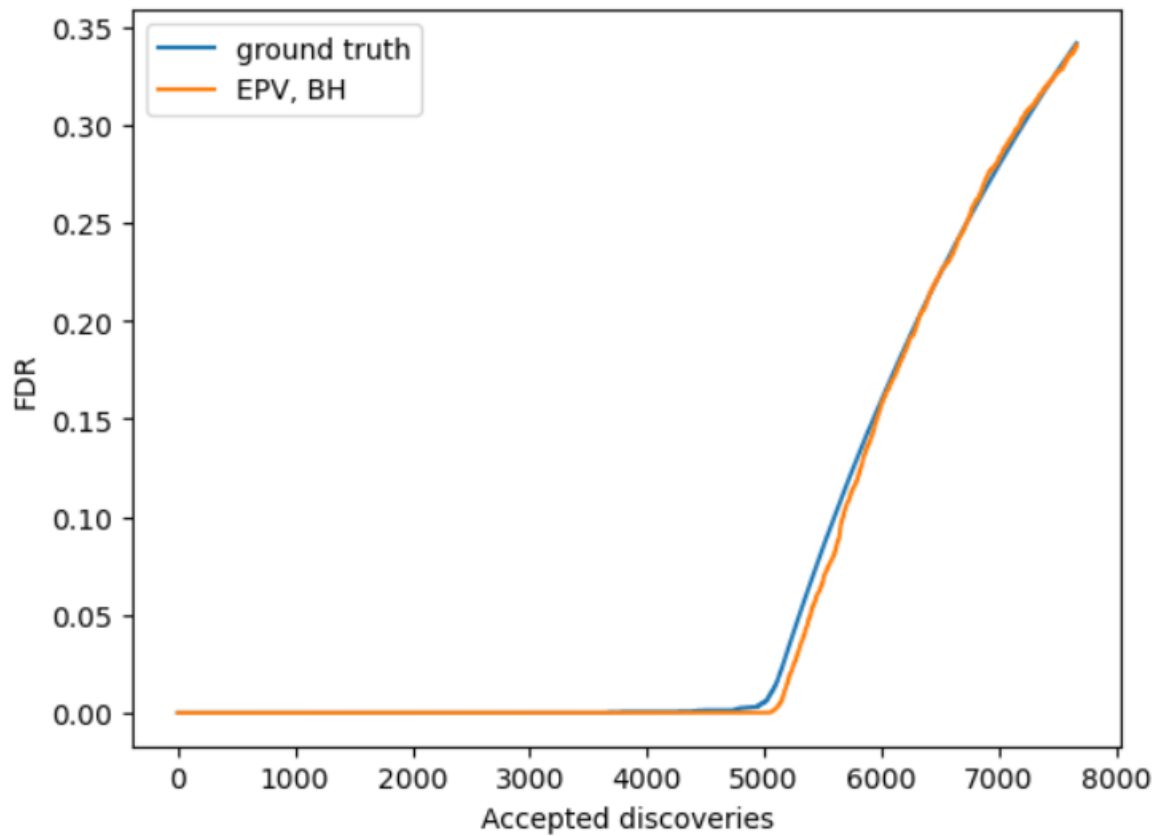


Figure 11.1 – Graph showing dependence between the number of accepted discoveries and FDR. Blue line represents FDR constructed with true labels. Orange line – with BHP and EPV.

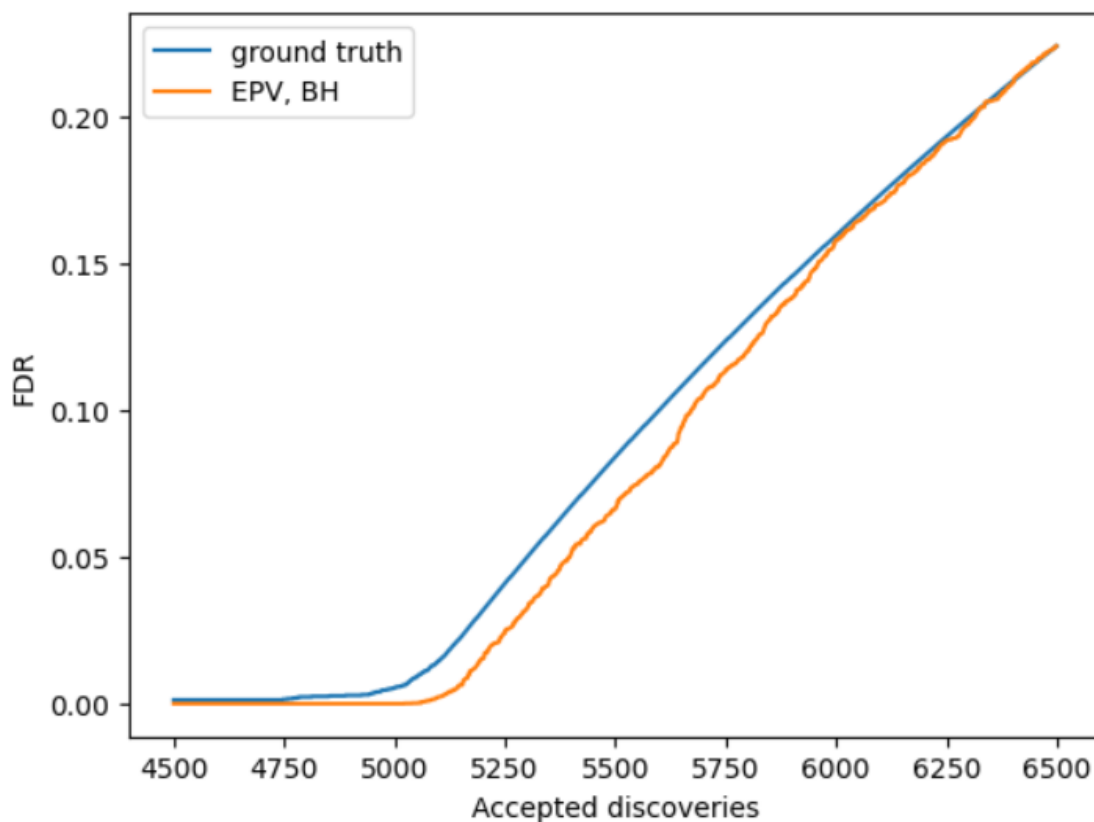


Figure 11.2 – Relevant part of the Figure 7.1 on a larger scale.

4.8 Interface

Originally it was supposed that the model for classification of optical coherence tomography images will become a part of a big Multifunctional Disease Identification Module, but in the end this Module remained purely theoretical, so instead we created interface in the form of simple telegram bot ([Figure 12](#)). The bot accepts OCT images or links to them and returns prediction of the class.

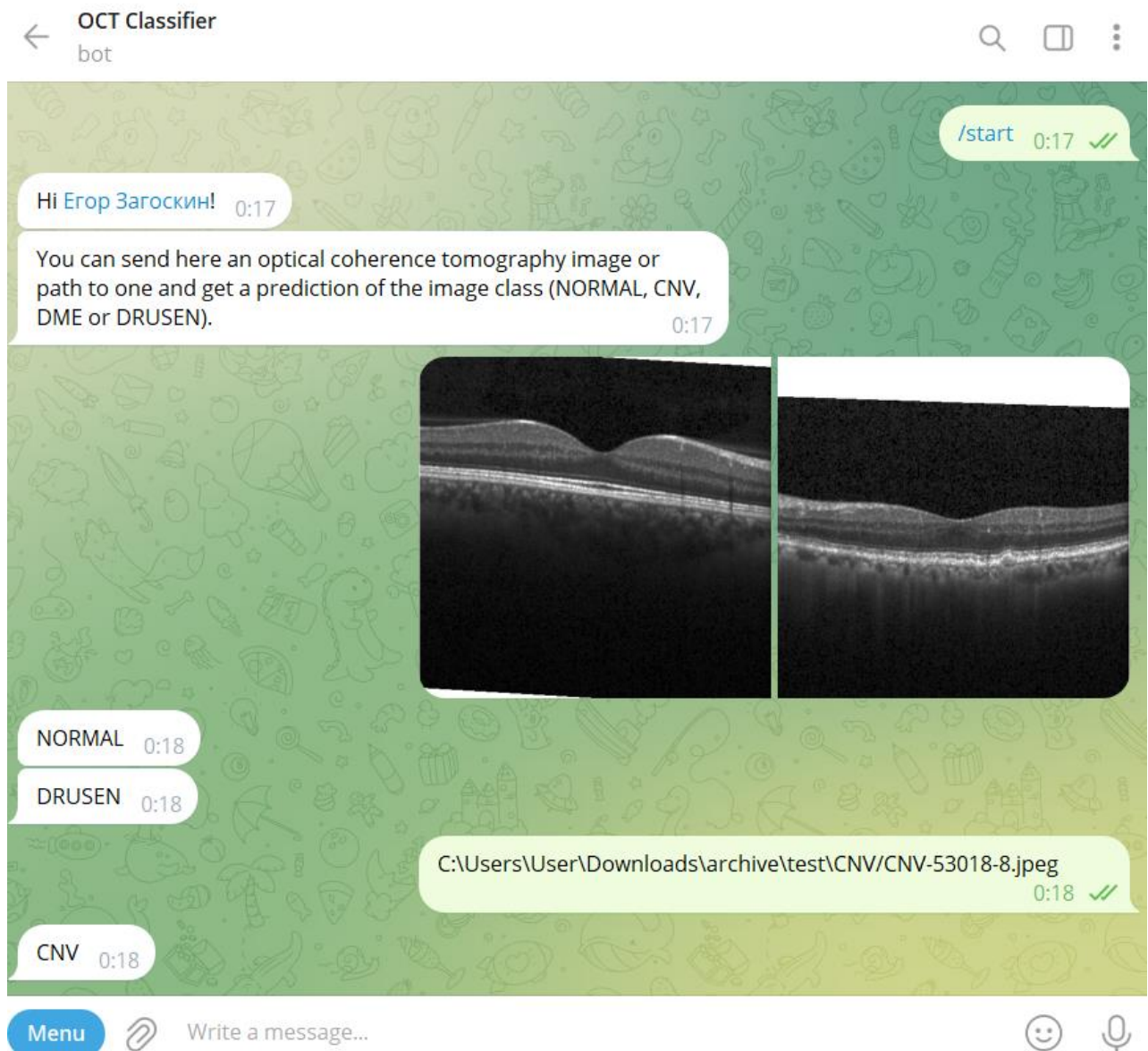


Figure 12 –Image showing an interaction with the telegram bot.

5 Conclusion

Overall, the goals of the project mostly were achieved. In the first part of the work, we successfully applied the planned methods (Optuna, Knowledge Distillation, Layer-wise Relevance Propagation, False Discovery Rate control) and exceeded the benchmarks for the PathMNIST.

In the second part of the work, we created a lightweight model outperforming the existing solution for the problem and provided in addition to this a method for FDR control and interface in the form of telegram bot. Second part of the project was affected by unexpected challenge of large data size relative to the available computing capacities. It made use of Optuna impractical (not enough resources to find the best possible model). It also made us to use most efficient models with complex structure which were not interpretable.

Possibilities for the future work include: creation of models for detection of other diseases using the constructed pipeline in order to create the true Multifunctional Disease Identification Module, improvement of interpretation methods so that they gave the results of better quality and were applicable for more complex models, advancement further into statistics to get better understanding of model performance beyond precision, recall and FDR.

References

1. Jiancheng Yang, Rui Shi, Donglai Wei, Zequan Liu, Lin Zhao, Bilian Ke, Hanspeter Pfister, Bingbing Ni. “MedMNIST v2 -- A large-scale lightweight benchmark for 2D and 3D biomedical image classification”. arXiv preprint, arXiv: 2110.14795, version 2. 2022.
2. Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, Masanori Koyama. “Optuna: A Next-generation Hyperparameter Optimization Framework”. arXiv preprint, arXiv: 1907.10902. 2019.
3. “Benchmarks with Kurobako”. Published in 2020. URL: <https://github.com/optuna/optuna/wiki/Benchmarks-with-Kurobako> (date of request: 01.02.2024).
4. Alexandros Chariton. “Knowledge Distillation Tutorial”. URL: https://pytorch.org/tutorials/beginner/knowledge_distillation_tutorial.html (date of request: 01.02.2024).
5. Wojciech Samek, Grégoire Montavon, Andrea Vedaldi, Lars Kai Hansen, Klaus-Robert Müller. “Explainable AI: Interpreting, Explaining and Visualizing Deep Learning”. Published in 2019 by Springer Link.
6. A. Borevsky. “Utilizing exact p-values for improved classification and accurate false discovery rate control”. Bachelor’s thesis. Higher School of Economics, 2023.
7. Fabrizio Ravelli. “Retinal OCT images”. URL: <https://www.kaggle.com/datasets/fabrizioravelli/retinal-oct-images-split> (date of request: 27.05.2024).
8. Mohamed Gobara. “96.1% in Retinal OCT CNN Model”. URL: <https://www.kaggle.com/code/mohamedgobara/96-1-in-retinal-oct-cnn-model> (date of request: 27.05.2024).
9. Mingxing Tan, Quoc V. Le. “EfficientNetV2: Smaller Models and Faster Training”. arXiv preprint, arXiv: 2104.00298. 2021.

Applications

Both parts of the work as well as the program implementing the telegram bot and the model used in the bot are available on GitHub: <https://github.com/Ezagoskin/Multifunctional-Disease-Identification-Module>